

BDF - Customized Product Documentation

```markdown

<div align="center">



</div>

## BDF Bakery Automation: Racer2 V3 Documentation

Welcome to the BDF-customized documentation for Racer2 V3, your advanced AI Core service designed to enhance line-scan acquisition, precise frame stitching, live streaming, and optional AI-assisted inspection for your bakery production lines. This document provides a professional and simple overview, focusing on how Racer2 V3 streamlines automation, improves production monitoring, and offers operator-friendly insights into your processes.

---

### 1. System Overview

Racer2 V3 is engineered to provide BDF with a powerful tool for monitoring and quality control in bakery automation. It integrates two high-speed line cameras to capture and stitch images of your products, offering real-time visual feedback and advanced AI capabilities.

#### 1.1 Purpose and Benefits for BDF

The primary goal of Racer2 V3 is to bring precision and intelligence to your bakery production. For BDF, this means:

- **Automated Product Monitoring:** Continuous acquisition of detailed images across your production line.
- **Seamless Visual Inspection:** Stitching images from multiple cameras into a single, comprehensive view of your products.
- **Real-time Insights:** Live streaming and readily available image captures for immediate operational feedback.
- **Enhanced Quality Control:** Optional AI-assisted inspection identifies anomalies and tracks product characteristics, ensuring BDF quality standards are met.
- **Flexible Operations:** Designed to adapt to different production setups, whether cameras are integrated directly or accessed remotely.

#### 1.2 Core Capabilities

- **Dual-Camera Acquisition:** Captures images from two dedicated cameras simultaneously.

- **Stitched Frame Generation:** Combines individual camera outputs into a single, wide image for complete product coverage.
- **Easy Image Access:** Browser-friendly JPEG capture and live video streaming through a web interface.
- **Adaptable Camera Access:** Supports direct camera connection, remote camera services, or simulated camera feeds for development and testing.
- **AI-Powered Inspection:** Optional AI models for advanced detection, tracking, and annotation of products and features.
- **Tracked-Object Color Analysis:** Monitors and records color information for specific product areas, with in-memory buffering for historical data.
- **External System Integration:** Ability to forward frames to other inference services or perform local file-based analysis.

### 1.3 How It Works: A High-Level Flow for BDF

1. **System Start:** The main application launches, ready to manage all operations.
2. **Camera Setup:** Based on your BDF setup, the system configures its camera access (direct, remote, or simulated).
3. **Image Capture:** The chosen camera setup begins capturing, polling, or generating product images.
4. **AI Readiness (Optional):** If AI inspection is enabled, the intelligence models load in the background, reporting their progress.
5. **Data Delivery:** The latest processed image of your products is made available through various access points:
  - For direct image retrieval.
  - For sending to external analysis systems.
  - For file-based AI analysis.
  - For live display in a browser or other applications.
6. **Color Analysis (Optional):** When active, the system analyzes and buffers color data for tracked objects, providing valuable production monitoring metrics.

---

## 2. System Architecture

The Racer2 V3 system for BDF bakery automation is structured to ensure reliability and performance. It consists of interconnected components designed for efficiency and ease of management.

<div align="center">

<!-- Placeholder for a diagram illustrating the runtime layers and their interactions -->

\*Placeholder: Diagram of System Layers\*

</div>

## 2.1 Operational Layers

The system is organized into three main layers, working together to deliver a comprehensive solution for BDF:

1. **Application Access Points:** These are the initial gateways to the system, launching the main monitoring application and, if needed, a dedicated camera service.

- The primary application, handling user interface, streaming, and AI tasks.
- A separate camera service for direct hardware control, useful for remote deployments.

2. **Core Services Layer:** This layer exposes the main functionalities, including the operator interface, live streaming, AI processing, and color analysis features.

- Manages the main web interface and core processing.
- Provides the camera-specific API when a dedicated camera service is used.

3. **Specialized Libraries Layer:** This layer contains the fundamental building blocks and specialized capabilities for vision, AI, and color analysis.

- Handles the Basler camera control and image stitching.
- Contains the AI models and processing logic.
- Manages color analysis and buffering.

## 2.2 Camera Integration

The main BDF application intelligently builds its camera runtime to match your specific setup:

- **Local Mode:**
  - The system directly controls your Basler cameras within the main application.
  - Ideal when the main application computer is physically connected to the cameras.
- **Remote Mode:**
  - The main application fetches images and alignment data from a separate camera service.
  - Perfect for setups where cameras need to be close to the hardware, while the main application runs on a different machine.
- **Emulated Mode:**
  - Generates synthetic frames internally for development, testing, and UI validation without needing physical cameras.
  - Useful for BDF engineers to develop and test new features offline.

The dedicated camera service (if used) supports only 'local' or 'emulated' camera modes, focusing solely on image acquisition without AI processing.

## 2.3 Camera Vision Pipeline

The live image acquisition and stitching process is a critical part of Racer2 V3, ensuring high-quality visuals for BDF's production monitoring.

<div align="center">

<!-- Placeholder for a diagram illustrating the camera pipeline -->

\*Placeholder: Diagram of Camera Pipeline\*

</div>

High-level flow for BDF's product images:

1. **Camera Discovery:** Identifies available Basler cameras on your production line.
2. **Camera Verification:** Ensures the required number of cameras are connected and ready.
3. **Camera Configuration:** Sets up camera triggers, pixel formats, and optional white balance, unless external Pylon configurations are specified.
4. **Strip Acquisition:** Captures multiple "strips" (thin slices) of your product from each camera.
5. **Vertical Stacking:** Stacks these strips vertically for each camera to form a complete product view.
6. **Offset Adjustment:** Applies horizontal and vertical adjustments for precise alignment.
7. **Image Stitching:** Combines the outputs from multiple cameras into one seamless, wide image of your product.
8. **Color Conversion:** Converts the image to a standard BGR format if the original camera output requires it.
9. **Image Processing:** Resizes and encodes the final image into a JPEG format.
10. **Caching:** Stores the latest JPEG image for quick access by monitoring screens and other applications.

Operational adjustments to camera alignment can be managed through dedicated system controls.

## 2.4 Remote Camera Operation

For BDF setups that benefit from separating camera hardware from the main application, the `remote` camera mode continuously retrieves necessary data from the dedicated camera service. This allows:

- The camera host to maintain direct, low-latency access to the Basler hardware.
- The main BDF application to consume processed frames over your network.
- The web interface and AI stack to be deployed independently, optimizing resource allocation.

## 2.5 AI Inspection Pipeline (Optional)

The AI pipeline is at the heart of Racer2 V3's advanced inspection capabilities for BDF, automating quality checks and providing valuable data.

<div align="center">

<!-- Placeholder for a diagram illustrating the AI pipeline -->

\*Placeholder: Diagram of AI Pipeline\*

</div>

The selected AI model is configured to best suit your inspection needs:

- **RF-DETR:** Utilizes advanced detection and tracking for precise product anomaly identification.
- **YOLO with SORT:** A robust combination for general object detection and continuous tracking of products or features.

Key operational aspects:

- **Asynchronous AI Startup:** AI initialization runs in the background, ensuring the main application starts quickly and doesn't delay production monitoring.
- **Status Monitoring:** The system status reports AI checkpoint progress, allowing BDF operators to know when AI is fully operational.
- **Graceful Operation:** Until AI is ready, the live stream will show raw frames, transitioning seamlessly once AI processing begins.
- **Offline Analysis:** A dedicated function supports inference on images stored on disk, useful for post-production analysis or re-evaluation.

#### #### 2.5.1 Color Analysis

Integrated within the AI pipeline is a sophisticated `ColorBuffer` that stores recent color history of tracked products or features. This feature provides BDF with:

- **Real-time Color Samples:** Latest color state for each identified object.
- **Rolling History:** Historical color data for trend analysis and consistency monitoring.
- **Data Export:** Ability to export color data in CSV format for reporting.
- **Production-Ready Output:** Structured color analysis data formatted for downstream consumption and integration into BDF's existing systems.

#### 2.6 Data Access for BDF Operators

The latest processed frame of your products is readily available for various BDF operational needs:

- **Direct Image Retrieval:** For individual frame inspection.
- **External Analysis:** For forwarding to other processing services.
- **File-Based Inference:** For local AI analysis of specific images.
- **Live Monitoring:** Through the WebSocket stream for continuous visual feedback.
- **Browser Interface:** Directly accessible via the user-friendly web interface.

---

### 3. Configuration

To ensure Racer2 V3 operates perfectly within your BDF bakery environment, the system relies on specific configuration settings. These settings are designed to be flexible, allowing you to tailor the system to your exact production monitoring and automation requirements.

#### 3.1 Configuration Sources

The project uses two primary methods for managing settings:

- **Operational Settings:** Non-sensitive runtime parameters that define how the system behaves (e.g., camera mode, AI model). These are typically stored in a `settings.json` file.
- **Security Credentials:** Sensitive information like API keys for AI services. These are kept secure in `.env` files.

#### 3.2 Loading Priority

Configuration values are loaded in a specific order, with later sources overriding earlier ones:

1. Initial values from a base `.env` file.
2. Any specific environment file (e.g., for debug mode) overrides the base `.env`.
3. `settings.json` is applied, providing default or profile-specific operational settings.
4. Crucially, any environment variable explicitly set in your operating system will override values from both `.env` files and `settings.json`.

In practice for BDF:

- Sensitive credentials should reside in `.env` for security.
- Operational choices (like camera mode) can be managed in `settings.json` or overridden by environment variables for quick adjustments.

#### 3.3 Key Operational Settings (`settings.json`)

The `settings.json` file supports different profiles (`production`, `debug`, `docker`) to suit various BDF deployment scenarios. Here are some key settings relevant to your operations:

##### #### Stream Settings

- `STREAM_DELAY_S`: Controls the delay between sending frames via the live stream, allowing BDF operators to adjust stream responsiveness.

##### #### External Inference (if used)

- `INFERENCE_URL`: The network address of an external AI analysis service, if BDF uses one.

##### #### Camera Runtime Configuration

These settings dictate how Racer2 V3 interacts with your BDF production line cameras:

- `CAMERA_MODE`:

- ``local``: The main application directly controls the Basler cameras. Ideal for a single, integrated BDF setup.
- ``remote``: The main application polls a separate camera service. Useful for distributed BDF setups.
- ``emulated``: Generates synthetic frames for testing and development without needing physical cameras.
- `CAMERA_WHITE_BALANCE`: Enables automatic one-shot white balance for consistent color representation of BDF products.
- `CAMERA_TRIGGER_SOURCE`, `CAMERA_TRIGGER_ACTIVATION`, `CAMERA_TRIGGER_SELECTOR`, `CAMERA_TRIGGER_MODE`: These advanced settings control how your cameras are triggered to capture images. For most BDF operations, the defaults are suitable.
- `CAMERA_PIXEL_FORMAT`: Defines how the camera images are interpreted for accurate color processing (e.g., `BGR8` for direct OpenCV compatibility, or `YCbCr422_8` for conversion).
- `CAMERA_CONFIG_BY_PYLON`: When ``true``, the system defers camera configuration to external Pylon software, allowing BDF specialists to manage camera settings independently.
- `REMOTE_CAMERA_BASE_URL`: The address of the dedicated camera service when `CAMERA_MODE` is ``remote``.
- `EMULATED_CAMERA_WIDTH`, `EMULATED_CAMERA_HEIGHT`, `EMULATED_CAMERA_FPS`: Define characteristics of generated frames in ``emulated`` mode.
- `EMULATED_CAMERA_IMAGE_PATH`: An optional file path for a specific image to be used as a source in ``emulated`` mode.

#### #### AI Runtime Configuration (Optional)

These settings control the AI inspection capabilities for BDF:

- `DETECTION_AND_TRACK`: Enables or disables AI object detection and tracking.
- `AI_MODEL_NAME`: Selects the AI model (e.g., `rfdet` for advanced detection, or another value for YOLO-based detection).
- Other `YOLO_MODEL_PATH`, `RFDETR_CONF_THRESH`, `CORE_MODEL_SAM_ENABLED` settings fine-tune specific AI model behaviors.

### 3.4 Security Credentials (`.env`` Files)

The `.env`` files are used to store sensitive information specific to your BDF environment, such as API keys for external services.

Example `.env`` file snippet:

```
``env
```

```
INFERENCE_API_HEADER=X-API-Key
```

```
INFERENCE_API_KEY=your-api-key-here
```

```
RFDETR_MODEL_ID=your-project/your-version
```

ROBOFLOW\_API\_KEY=your-roboflow-api-key

...

### 3.5 Camera Library Configuration

The underlying camera library has its own detailed configuration. Most fields are set automatically by the application based on your `CAMERA\_\*` settings. Key fields influencing BDF's operation include:

- `num\_cameras`: The number of Basler cameras in use.
- `pixel\_format`: How camera buffers are interpreted (e.g., `BGR8` for direct processing or `YCbCr422\_8` for conversion).
- `trigger\_mode`: Whether the camera uses hardware triggering.
- `white\_balance`: Enables one-shot white balance for consistent lighting.
- `is\_config\_by\_pylon`: Determines if the application or Pylon Viewer controls camera feature writing.
- `h\_offset`, `v\_offset`: Horizontal and vertical offsets for fine-tuning image stitching.

### 3.6 AI Library Configuration

The AI library also has its own configuration, drawing from your application settings. Important fields include:

- `model\_name`: The selected AI model.
- `roboflow\_api\_key`: For accessing Roboflow-based models.
- `color\_analyze\_enabled`: Determines if color analysis is active for tracked objects.

### 3.7 Recommended Practice for BDF Operators

- **Secure Credentials:** Always keep sensitive API keys and other secrets exclusively in `.env` files.
- **Operational Control:** Use `settings.json` for non-sensitive, environment-specific operational settings.
- **Debug Mode:** For testing or development, use `APP\_ENV=debug` instead of modifying production settings directly.
- **Docker Settings:** Reserve the `docker` profile for container-specific settings that differ from your local or production host environments.

---

## 4. API Reference (Simplified for BDF Operators)

Racer2 V3 provides a set of interfaces (APIs) that allow other systems or custom tools to interact with your BDF bakery automation system. This section simplifies these interfaces, focusing on their function for monitoring and data retrieval.

### 4.1 Main Application Access

By default, the main application is available at:

```text

http://localhost:8000

...

The dedicated camera service (if deployed separately) is at:

```text

http://localhost:8001

...

## 4.2 Key Operational Endpoints

- **Get Operator Interface ('GET /')**
- **Purpose:** Returns the main web interface for BDF operators to view live streams and system status.
- **Returns:** The HTML dashboard for your browser.
- **Get System Status ('GET /status')**
- **Purpose:** Provides a comprehensive overview of the system's current health, including AI initialization, camera readiness, and stream status. Essential for BDF operators to quickly diagnose issues.
- **Returns:** A summary of AI, camera, and stream states.
- **Example:** `ai.phase` might show "ready" or "loading," and `camera.ready` will indicate if your production line cameras are active.
- **Get Latest Product Image ('GET /capture')**
- **Purpose:** Retrieves the most recent JPEG image from the camera system. Useful for BDF operators needing a quick snapshot of the production line.
- **Returns:** A JPEG image.
- **Note:** May return an error if cameras are not ready or no image has been processed yet.
- **Forward for External Analysis ('POST /analyze')**
- **Purpose:** Sends the latest captured frame to an external AI inference service for specialized analysis (if configured).
- **Returns:** Analysis results from the external service.
- **Note:** Handles retries if the external service is busy.
- **Get Camera Alignment Settings ('GET /camera/alignment')**
- **Purpose:** Retrieves the current settings for how images from multiple cameras are stitched together.
- **Returns:** The current horizontal and vertical offsets, and the number of images being stitched.
- **Update Camera Alignment Settings ('POST /camera/alignment')**
- **Purpose:** Allows BDF specialists to adjust the stitching alignment (horizontal and vertical offsets) and reset the image buffer.
- **Request:** Provide new `h\_offset`, `v\_offset`, or optionally `num\_images`.
- **Returns:** The updated alignment settings, confirming the change.
- **Get Color Analysis Data ('GET /color-data')**

- **Purpose:** Retrieves the buffered color analysis data for tracked products/objects and then clears the buffer. Provides BDF operators with historical color information for quality control.
- **Returns:** A list of objects with their latest color samples and history.
- **Note:** Will be empty if AI detection/tracking is disabled or no objects have been processed.
- **Perform Local AI Inference (POST /inference)**
- **Purpose:** Runs the configured AI model on a specific image file located on the system's disk. Useful for BDF engineers to test AI models on pre-recorded images.
- **Request:** Specify the image file name and its directory.
- **Returns:** AI analysis results, including a path to an annotated image.
- **Note:** Requires the image file to be accessible on the machine running the main application.
- **Stream Live Images (WebSocket /ws)**
- **Purpose:** Provides a continuous stream of JPEG images to connected clients (like your browser). This stream can show either raw frames or AI-annotated frames, depending on AI readiness.
- **Provides:** Live JPEG frames, or text messages like "warming\_up" during system initialization or camera issues.

---

## 5. Troubleshooting for BDF Operators

This section provides practical guidance for BDF operators to diagnose and resolve common issues with the Racer2 V3 system, focusing on straightforward solutions.

### 5.1 Server Starts But No Frames Arrive

**Symptoms:** The system seems to be running, but no images appear in the web interface or `/capture` endpoint.

#### Checks:

- **System Status:** Check the `/status` endpoint (or the dashboard).
- Is `camera.ready` reported as `false`?
- Is the live stream showing text messages like `warming\_up` instead of images?

#### Solutions based on Camera Mode:

- **Local Mode:**
- Verify that `pyppylon` (the camera driver software) is correctly installed.
- Ensure both Basler cameras are physically connected and powered on.
- Confirm that the system's user account has the necessary permissions to access the cameras.
- **Remote Mode:**
- Verify the `REMOTE\_CAMERA\_BASE\_URL` setting for any typos.
- Confirm that the separate camera service is running on its host machine (usually on port `8001`).
- Test directly: Can you access `/capture` and get a JPEG image?

- **Emulated Mode:**
- Ensure `CAMERA\_MODE` is set to `emulated`.
- If `EMULATED\_CAMERA\_IMAGE\_PATH` is specified, verify that the image file exists and is readable.

## 5.2 Camera Busy or Unavailable

**Symptoms:** The `/status` endpoint reports the camera is not ready, or logs show errors about camera initialization failure or busy hardware.

### Checks:

- **External Software:** Close any other applications that might be using the Basler cameras, such as Pylon Viewer.
- **Duplicate Processes:** Confirm that no other instance of the Racer2 V3 service is still running in the background.
- **System Restart:** If the camera driver seems stuck, a restart of the camera host machine might resolve the issue.

## 5.3 Remote Camera Mode Not Refreshing

**Symptoms:** In remote mode, the system is connected to the camera host, but frames are not updating.

### Checks:

- **URL Accuracy:** Double-check `REMOTE\_CAMERA\_BASE\_URL` for any typos.
- **Port Accessibility:** Confirm that the camera host's port `8001` is open and accessible from the main application's machine.
- **Camera Host Status:** Check `GET <REMOTE\_CAMERA\_BASE\_URL>/status` directly on the camera host; it should return a `200` status.
- **Network Connectivity:** Ensure the main application's environment can successfully communicate with the camera host (e.g., no Docker networking issues, no Windows firewall blocks).
- **Remote `/capture` Failure:** Even if the camera host is up, check if its `/capture` endpoint is failing.

## 5.4 Live Stream Shows Text Instead of Images

**Symptoms:** The WebSocket stream or browser interface displays messages like `warming\_up` or camera error text instead of live product images.

**Explanation:** This is expected behavior when:

- The camera system is starting up or calibrating.
- Camera initialization has encountered an error.
- In remote mode, the system is waiting for the first valid frame from the camera service.

The text messages provide helpful diagnostic information to BDF operators.

### 5.5 AI Never Becomes Ready

**Symptoms:** The `/status` endpoint indicates that `ai.phase` is not `ready` or remains in a `pending` or `active` state for a prolonged period.

#### Checks:

- **Status Endpoint:** Verify `GET /status` to see AI checkpoint progress.
- **Credentials:** Ensure your `.env` file contains valid Roboflow or other AI service credentials.
- **Model Selection:** Confirm that the `AI_MODEL_NAME` in your configuration is the intended and supported model for your BDF quality inspection.
- **Initialization Time:** Large AI models can take significant time to load. Allow for this during startup.

### 5.6 `POST /inference` Returns 404

**Symptoms:** When attempting to run AI inference on a file, the system returns a `404` error.

#### Checks:

- **Directory Path:** Verify that the `SrcDir` parameter points to an existing directory on the machine where the main application is running.
- **Image Name:** Ensure the `ImageName` parameter is correct and matches an image file within `SrcDir`.
- **Combined Path:** Confirm that the full path `SrcDir/ImageName` genuinely exists on the disk.
- **Local Access:** Remember, this endpoint reads files directly from the local disk of the main application, not from a remote network location.

### 5.7 Color Endpoints Return Empty Data

**Symptoms:** Requests to `/color-data` or `/color-analysis` return empty object lists.

#### Possible Reasons:

- **Detection/Tracking Disabled:** `DETECTION_AND_TRACK` might be set to `false` in your configuration.
- **AI Not Ready:** The AI model needs to be fully loaded (`ai.phase=ready`) to process objects for color analysis.
- **No Tracked Objects:** No products or features have been detected and tracked since the last buffer clear.
- **Buffer Already Cleared:** A previous call to `/color-data` or `/color-analysis` automatically clears the buffer after returning data.

### 5.8 Pixel Format Problems

**Symptoms:** Output frames show incorrect colors, or errors appear during capture related to color conversion (e.g., OpenCV YUV conversion errors).

#### Checks:

- **Matching Format:** Verify that `CAMERA\_PIXEL\_FORMAT` in your configuration accurately matches the actual pixel format output by your Basler cameras.
- **External Configuration:** If you use Pylon Viewer or other external tools to configure cameras, set `CAMERA\_CONFIG\_BY\_PYLON=true` to prevent the application from overwriting those settings.
- **BGR Output:** If your cameras are already configured to output OpenCV-ready BGR frames, ensure `BGR8` or `BGR8Packed` is selected for `CAMERA\_PIXEL\_FORMAT`.

## 5.9 General Diagnostic Sequence for BDF Operators

When encountering an issue, follow this systematic approach:

1. **Check System Status:** Call `GET /status` to get an overall health report.
2. **Verify Camera Readiness:** Confirm that `camera.ready` is `true`.
3. **Confirm Image Capture:** Check `GET /capture` to ensure it returns a JPEG image.
4. **Validate Web Interface:** Verify that the browser interface (`GET /`) loads and updates with live images.
5. **AI Readiness (If Enabled):** If AI is active, wait for `ai.phase` to report `ready`.
6. **Test AI/Color Functions:** Only then, test `POST /analyze`, `POST /inference`, or WebSocket streaming for AI-annotated frames, and color analysis endpoints.

---

## 6. Deployment Options for BDF

Racer2 V3 offers flexible deployment options to suit various BDF bakery automation setups. Whether you need a single, integrated system or a distributed architecture, these guidelines will help you deploy efficiently.

### 6.1 Baseline Setup for All Deployments

Before proceeding with any deployment option, ensure these steps are completed:

- **Virtual Environment:** Create a Python virtual environment to manage dependencies.
- **Install Dependencies:** Install all required libraries using `pip install -r requirements.txt`.
- **Environment Variables:** Create your `.env` file from the provided `.env.example` template and populate it with your specific BDF credentials and settings.
- **Review Settings:** Check `settings.json` to configure operational parameters.
- **Profile Selection:** Confirm the desired operational profile: `production`, `debug`, or `docker`.

### 6.2 Option 1: Single-Process Local Deployment (Integrated BDF Line)

This is the simplest setup, ideal when the same Windows machine has direct physical access to your Basler cameras and handles all processing.

```
""powershell
```

```
python .\main.py
```

```
'''
```

#### **Recommended BDF Settings:**

- `CAMERA\_MODE=local`: The main application directly manages the cameras.
- `DETECTION\_AND\_TRACK=true` or `false`: Depending on whether AI inspection is required for this line.

**When to Use:** This setup is perfect for BDF production lines where:

- Pylon camera drivers are installed on the local machine.
- The host machine is designated to handle both image acquisition and AI processing.
- Network separation between camera and processing is not a concern.

#### **6.3 Option 2: Debug or Hardware-Free Deployment (BDF Development/Testing)**

Use this option for BDF development, UI testing, API validation, or AI logic testing without needing physical cameras.

```
'''powershell
```

```
$env:APP_ENV="debug"
```

```
python .\main.py
```

```
'''
```

#### **Recommended BDF Settings:**

- `CAMERA\_MODE=emulated`: Generates synthetic frames internally.
- Optional `EMULATED\_CAMERA\_IMAGE\_PATH`: Use a specific image file for consistent testing.

#### **6.4 Option 3: Split Camera and Application Deployment (Distributed BDF Lines)**

This option is valuable when the camera host needs to remain physically close to the Basler hardware on the production line, while the web interface and AI processing can run on a separate, more powerful machine.

##### **#### 6.4.1 Camera Host Setup**

On the machine directly connected to the Basler cameras:

```
'''powershell
```

```
python .\camera_main.py
```

```
'''
```

This will start the dedicated camera-only service, typically listening on port `8001`.

#### #### 6.4.2 Main Application Host Setup

On the machine where the main BDF application (web and AI) will run:

- **Configure:** Set `CAMERA\_MODE=remote` in your `settings.json` or environment variables.
- **Point to Camera Host:** Set `REMOTE\_CAMERA\_BASE\_URL=http://<camera-host>:8001`, replacing `<camera-host>` with the actual IP address or hostname of your camera host.

Then, start the main application:

```
``powershell

python .\main.py
``
```

#### 6.5 Option 4: Docker for the Main Application (Containerized BDF Deployment)

The provided `docker-compose.yml` is primarily intended for containerizing the main BDF application, not for direct Basler camera capture.

##### Recommended Pattern for BDF:

- **Windows Host:** Run `camera\_main.py` directly on a Windows machine that has Basler camera access.
- **Docker Container:** Deploy the main BDF application within a Docker container.
- **Remote Mode:** Configure the main application in the container to use `CAMERA\_MODE=remote`.
- **Access Host:** Set `REMOTE\_CAMERA\_BASE\_URL=http://host.docker.internal:8001` within the Docker container's configuration. `host.docker.internal` allows the container to communicate with services running on the host machine.

To start the Dockerized main application:

```
``powershell

docker compose up --build
``
```

#### 6.6 Environment Expectations for BDF

##### #### Local Camera Mode

- **Pylon Installation:** Ensure Basler Pylon drivers and SDK are correctly installed.
- **Camera Access:** The application process must have permissions to access both cameras.
- **Configuration Match:** Trigger and pixel format settings must align with your physical camera setup.

##### #### Remote Camera Mode

- **Camera Host API:** The camera host must be running and exposing `/status`, `/capture`, and `/camera/alignment` endpoints.
- **Network Stability:** Stable network connectivity between the main application and the camera host is crucial for continuous frame updates.
- **Frame Freshness:** The timeliness of frames depends on the polling interval configured for the remote connection.

#### #### Emulated Mode

- **No Hardware Needed:** This mode requires no physical camera hardware, making it ideal for software-only development and testing.
- **Safe Testing:** It's the safest choice for frontend and integration testing without impacting live production.

### 6.7 Verification Checklist After BDF Deployment

After any deployment, perform these checks to confirm proper operation:

- **System Status:** Access `GET /status` and verify it returns a `200` (OK) status.
- **Camera Ready:** Confirm that `camera.ready` reports `true` for your chosen camera mode.
- **Image Capture:** Check `GET /capture` to ensure it successfully returns a JPEG image.
- **Web Interface:** Load the browser page (`GET /`) and confirm it updates with live images.
- **Live Streaming:** Verify that WebSocket streaming (`/ws`) is functional.
- **AI Readiness (If Enabled):** If AI is part of your BDF setup, confirm that `ai.phase` eventually becomes `ready`.
- **Remote Camera Service:** If using remote mode, verify that the dedicated camera service on port `8001` also returns a healthy `GET /status`.

### 6.8 Operational Notes for BDF

- **Fault Tolerance:** The system is designed to be resilient. It will continue to run for diagnostics even if cameras are initially unavailable during startup.
- **AI Startup Time:** Be aware that AI model initialization may take longer than the initial server startup, especially for larger models.
- **Local Inference Path:** The `POST /inference` endpoint requires that the specified image files are physically accessible on the machine running the main BDF application.

...